

Cómputo Concurrente 2026

Práctica 8

De Java a Rust y Clojure: Paradigmas de Concurrency

Profa: Gilde Valeria Rodríguez

A lo largo del curso hemos razonado sobre corrección en programas concurrentes: linealizabilidad, consistencia secuencial, condiciones de progreso y estructuras lock-free. Todo ese análisis se ha hecho en Java. Sin embargo, los mismos problemas, y sus soluciones, se expresan de maneras radicalmente distintas en otros lenguajes. Esta práctica compara tres lenguajes que representan tres filosofías de diseño distintas ante el mismo problema: cómo lidiar con el estado compartido y el tiempo.

Una dimensión que el curso no ha explorado explícitamente es que las *data races*, ese fenómeno que en la Práctica 1 vimos producir resultados incorrectos, no son siempre un error. De hecho, algunas de las estructuras más eficientes que existen en la literatura explotan deliberadamente el comportamiento relajado de la memoria para lograr escalabilidad. Veremos un ejemplo concreto del libro de Herlihy & Shavit [HS08].

1 Contexto

1.1 Java: el modelo clásico de memoria compartida

Java nació en la era de los hilos del sistema operativo. Su modelo de concurrencia está basado en memoria compartida: todos los hilos viven en el mismo proceso y acceden al mismo *heap*. Cada hilo tiene su propia *stack* con variables locales, pero los objetos creados con `new` residen en el heap y son “visibles” para todos.

- **Gestión de memoria:** Heap compartido + stacks por hilo. Recolector de basura (GC) automático, evita fugas pero puede introducir pausas *stop-the-world*.
- **Sincronización:** Candados implícitos (`synchronized`) o explícitos (`ReentrantLock`). El programador protege secciones críticas manualmente.
- **Volatile y fences:** La palabra clave `volatile` garantiza visibilidad y establece barreras de memoria que impiden que el compilador o la CPU reordenen instrucciones. No garantiza atomicidad.
- **Tipos atómicos:** `AtomicInteger`, `AtomicLong`, `AtomicReference`, los mismos que usamos en las Prácticas 5 y 6 para operaciones CAS.

El modelo de memoria de Java (JMM), especificado en el JSR-133 [Manson et al., POPL 2005], define formalmente qué valores puede leer un hilo de una variable escrita por otro. El JMM es un ejemplo de modelo de memoria débil: el hardware moderno puede reordenar instrucciones y mantener cachés locales, y el JMM define exactamente cuándo esos reordenamientos son visibles, es el mismo tipo de modelo que estudia *Herdning Cats* [Alglave et al., 2014].

1.2 Rust: concurrencia sin miedo (*Fearless Concurrency*)

Hola mundo en: <https://github.com/gilde-valeria/gilde-valeria.github.io/blob/main/teaching/practicas/concurrencia-lenguajes/de-java-a-rust.org>

Rust ataca el problema desde la raíz: el compilador rechaza programas con *data races*. Esto se logra sin un recolector de basura, a través de un sistema de tipos que rastrea en tiempo de compilación quién posee cada dato y cuándo se puede acceder a él.

- **Gestión de memoria:** Heap y Stack, sin GC. El sistema de *Ownership* (propiedad) y *Borrowing* (préstamo) garantiza que la memoria se libera exactamente cuando deja de usarse, sin pausas ni fugas.
- **La regla de oro:** En cualquier punto del programa, o existe *una* referencia mutable (`&mut T`), o existen *múltiples* referencias inmutables (`&T`), pero nunca ambas al mismo tiempo. Esta regla hace imposibles los data races, si el código compila, no tiene data races.
- **Mutex acoplado al dato:** `Mutex<T>` en Rust no solo protege el acceso, el dato vive *dentro* del Mutex. No puedes leer el dato sin antes adquirir el lock. En Java, el programador decide qué proteger; en Rust, el compilador lo impone.
- **Compartir entre hilos:** `Arc<T>` (Atomic Reference Counter), análogo a `AtomicReference` en Java, permite que múltiples hilos posean una referencia al mismo dato.
- **Ordenamiento de memoria explícito:** `std::sync::atomic` expone `Ordering::SeqCst`, `Ordering::Acquire`, `Ordering::Release`, el mismo vocabulario de barreras de memoria que *Herding Cats* formaliza.

Rust es el lenguaje preferido para programación de sistemas de bajo nivel donde el rendimiento y la ausencia de GC son críticos: sistemas operativos, drivers, contratos inteligentes en blockchain.

1.3 Clojure: estado, identidad y el tiempo

Hola mundo en: <https://github.com/gilde-valeria/gilde-valeria.github.io/blob/main/teaching/practicas/concurrencia-lenguajes/de-java-a-clojure.org>

Clojure corre sobre la JVM, por lo que hereda el heap, los stacks y el recolector de basura de Java. Sin embargo, su filosofía de concurrencia es radicalmente distinta: en lugar de proteger el acceso a la memoria, elimina el estado mutable por defecto.

- **Inmutabilidad por defecto:** Las estructuras de datos de Clojure (listas, vectores, mapas) son persistentes e inmutables. Modificar un vector no destruye el original, crea una nueva versión compartiendo estructura con el anterior (*persistent data structures*).
- **Separación de identidad y estado:** Clojure distingue entre la *identidad* (la referencia, ej. 'la cuenta de Alice') y el *estado* (el valor en un momento dado, ej. '1 000 pesos'). Los `ref`, `atom` y `agent` son tipos de identidad con diferentes semánticas de actualización.
- **Para qué se usa en la industria:** Nubank lo usa para su core bancario. Se usa en motores de emparejamiento (*matching engines*), procesamiento de eventos en tiempo real, y pipelines de datos. El STM brilla cuando necesitas actualizar múltiples variables coordinadamente sin riesgo de deadlock.

Software Transactional Memory (STM)

¿Qué es STM? Es un modelo de concurrencia que traslada la abstracción de las transacciones de base de datos al acceso a memoria. En lugar de usar candados explícitos, el programador declara un bloque transaccional (`dosync` en Clojure). Dentro del bloque, todas las lecturas y escrituras a `refs` se registran en un log local al hilo. Al terminar el bloque, el runtime verifica si algún otro hilo modificó los mismos `refs` durante la ejecución. Si hubo conflicto, descarta el log y reintenta la transacción desde cero. Si no hubo conflicto, aplica todos los cambios atómicamente.

Las propiedades que garantiza el STM de Clojure son análogas a las propiedades ACID de una base de datos, pero en memoria:

Propiedad	Significado en STM	Equivalente en el curso
Atomicidad	Todos los cambios ocurren o ninguno	Punto de linearización único
Consistencia	El estado resultante respeta los invariantes	Objeto correcto
Aislamiento	Transacciones conc. no se ven entre sí	Operaciones no superpuestas
Durabilidad*	*No garantizada — STM es en RAM	N/A

Tabla 1: Propiedades del STM de Clojure y su equivalente en el curso.

La diferencia crucial frente a los candados: con STM no es posible un deadlock causado por el orden de adquisición de locks, porque no existen locks explícitos. La resolución de conflictos la hace el runtime, no el programador.

2 Data Races útiles: cuando la relajación es correcta

En la Práctica 1 vimos que un contador compartido sin sincronización produce resultados incorrectos. En la Práctica 5 analizamos los modelos de consistencia que formalizan exactamente qué significa ‘incorrecto’. Sin embargo, Herlihy y Shavit argumentan que existen situaciones donde permitir intencionalmente que un hilo lea un valor ‘obsoleto’ o ‘inconsistente’ es no solo aceptable, sino la base de implementaciones correctas y eficientes.

2.1 El contador de Consistencia Quiescente

El siguiente ejemplo, adaptado directamente de Herlihy & Shavit, muestra un contador concurrente que *no* es linealizable ni secuencialmente consistente, pero sí es *quiescentemente consistente*, y eso es suficiente para muchas aplicaciones de alto rendimiento.

```
1 import java.util.concurrent.atomic.AtomicLong;
2
3 /**
4  * Adaptado de Herlihy & Shavit, The Art of Multiprocessor Programming,
5  * Capítulo 5. Contador con Consistencia Quiescente.
6  *
7  * ADVERTENCIA INTENCIONAL: fetch() puede devolver un valor desactualizado
8  * mientras hay incrementos en vuelo. La "data race" es deliberada:
9  * el hilo lector puede ver un valor que no refleja los incrementos locales
10  * de los otros hilos. Si todos los hilos cesan (quiescence), el valor
11  * sera correcto.
12  *
13  * Propiedad: Quiescent Consistency (no Linearizable, no Seq. Consistent).
```

```

14  */
15  public class QCSCounter {
16      private final AtomicLong globalCount = new AtomicLong(0);
17      private final int threshold;
18
19      // La "data race" util: cada hilo tiene su copia local (stack privado).
20      // No hay sincronización entre el ThreadLocal y globalCount
21      // durante los incrementos -- eso es intencional.
22      private final ThreadLocal<Long> local =
23          ThreadLocal.withInitial(() -> 0L);
24
25      public QCSCounter(int threshold) { this.threshold = threshold; }
26
27      public void increment() {
28          long current = local.get() + 1;
29          if (current >= threshold) {
30              globalCount.addAndGet(current); // volcado atómico al global
31              local.set(0L);
32          } else {
33              local.set(current); // sin sincronización -- data race
34                                  // deliberada
35          }
36
37      }
38
39      // fetch() puede ser incorrecto durante ejecución concurrente
40      // (ignora los contadores locales de otros hilos).
41      public long fetch() { return globalCount.get(); }
42  }

```

Listing 1: Contador quiescentemente consistente.

Nota: La ‘data race’ aquí no es un bug, es una decisión de diseño consciente. El análisis de corrección formal dice: este contador satisface Consistencia Quiescente pero no Linealizabilidad ni Consistencia Secuencial. Para aplicaciones donde se consulta el contador solo en periodos de inactividad (ej. estadísticas periódicas, métricas de rendimiento), esto es completamente correcto y drásticamente más rápido que un `AtomicLong` puro bajo alta contención.

2.2 ¿Por qué es más rápido?

El costo de `AtomicLong.incrementAndGet()` bajo alta contención es alto: todos los núcleos intentan invalidar la misma línea de caché (*cache line bouncing*). El contador quiescente distribuye el trabajo: cada hilo escribe en su propia memoria (su stack, a través del `ThreadLocal`) y solo ocurre contención en el volcado al global, que sucede una vez cada `threshold` incrementos. El costo de 1000 incrementos pasa de $O(1,000)$ operaciones atómicas a $O(1,000/\text{threshold})$ operaciones atómicas.

Esta es exactamente la idea detrás del **SloppyCounter** de la Práctica 5 y del **árbol de difracción de Shavit**: relajar la consistencia para ganar escalabilidad. La pregunta correcta no es ‘¿hay una data race?’ sino ‘¿qué modelo de consistencia necesita mi aplicación?’

3 Ejemplos Aplicados

3.1 ¿Qué es un Smart Contract?

Un contrato inteligente (*smart contract*) es un programa que reside en una cadena de bloques (blockchain) y se ejecuta automáticamente cuando se cumplen condiciones predefinidas, sin necesidad de un intermediario. El código y sus resultados son inmutables y verificables públicamente. En términos de cómputo concurrente, un contrato inteligente es un objeto compartido accedido por múltiples transacciones que ocurren simultáneamente — exactamente el escenario que hemos analizado en el curso.

La conexión con el curso es directa: una blockchain debe garantizar que el estado global (el registro de balances) sea linealizabile — cada transacción debe parecer que ocurrió en un único instante de tiempo real. Si no fuera linealizabile, sería posible gastar el mismo saldo dos veces (el famoso *double-spend problem*).

¿Por qué Rust? Redes como Solana o Polkadot usan Rust para sus contratos inteligentes porque: (1) compila a WebAssembly (Wasm) o bytecode determinista, (2) no tiene Garbage Collector (las pausas GC son inaceptables cuando cada instrucción cuesta *gas*), y (3) el sistema de tipos garantiza ausencia de data races en tiempo de compilación — propiedad crítica cuando el contrato maneja valor real.

3.2 Contrato inteligente de tokens en Rust

```
1 use std::collections::HashMap;
2 use std::sync::{Arc, Mutex};
3 use std::thread;
4
5 /// Estado del contrato: un mapa de direcciones a balances.
6 /// El dato vive DENTRO del Mutex -- no se puede acceder sin el lock.
7 /// Arc permite que multiples hilos (transacciones) compartan la referencia
8
9 struct TokenContract {
10     balances: Mutex<HashMap<String, u64>>,
11 }
12
13 impl TokenContract {
14     fn new() -> Self {
15         let mut state = HashMap::new();
16         state.insert("Alice".to_string(), 1000);
17         state.insert("Bob".to_string(), 500);
18         TokenContract { balances: Mutex::new(state) }
19     }
20
21     /// Punto de linearizacion: el momento en que lock() retorna.
22     fn transfer(&self, from: &str, to: &str, amount: u64) -> bool {
23         let mut b = self.balances.lock().unwrap();
24         let from_bal = b.get(from).copied().unwrap_or(0);
25         if from_bal < amount {
26             println!("[RECHAZADA] {} no tiene fondos", from);
27             return false;
28         }
29         *b.entry(from.to_string()).or_insert(0) -= amount;
30         *b.entry(to.to_string()).or_insert(0) += amount;
31         println!("[OK] {} -> {} : {} tokens", from, to, amount);
32         true
33     }
34     // El Mutex se libera automaticamente al salir del scope (drop).
35     // Equivale al bloque finally { lock.unlock(); } de Java,
36     // pero garantizado por el compilador.
37 }
```

```

37     fn balance(&self, addr: &str) -> u64 {
38         self.balances.lock().unwrap().get(addr).copied().unwrap_or(0)
39     }
40 }
41
42 fn main() {
43     let contract = Arc::new(TokenContract::new());
44     let mut handles = vec![];
45
46     for _ in 0..3 {
47         let c = Arc::clone(&contract);
48         handles.push(thread::spawn(move || {
49             c.transfer("Alice", "Bob", 100);
50         }));
51     }
52     for h in handles { h.join().unwrap(); }
53
54     println!("Balance Alice: {}", contract.balance("Alice"));
55     println!("Balance Bob:   {}", contract.balance("Bob"));
56 }

```

Listing 2: Contrato inteligente de tokens en Rust. Compilar: `rustc --edition 2021 contrato.rs && ./contrato`

3.3 Transacciones bancarias en Clojure (STM)

El problema clásico de los candados para transferencias bancarias: para transferir de la cuenta *A* a la cuenta *B*, hay que bloquear *A*, luego bloquear *B*. Si otro hilo bloquea *B* primero y luego *A*, hay deadlock. Clojure elimina este problema completamente: no existen locks explícitos.

```

1 (ns banco.core)
2
3 ;; 'ref' crea una identidad coordinable via STM.
4 ;; @cuenta-alice dereferencia el ref para leer el valor actual.
5 (def cuenta-alice (ref 1000))
6 (def cuenta-bob   (ref 500))
7
8 (defn transferir
9     "Transfiere cantidad de origen a destino de forma atomica.
10     Si dos hilos colisionan, el STM reintenta automaticamente.
11     No es posible deadlock: no hay locks, solo transacciones STM."
12     [origen destino cantidad]
13     (dosync ; abre la transaccion STM
14         (let [saldo @origen]
15             (if (>= saldo cantidad)
16                 (do
17                     ;; alter aplica una funcion al ref dentro de la transaccion.
18                     ;; Si hay conflicto al commit, dosync se reintenta.
19                     (alter origen - cantidad)
20                     (alter destino + cantidad))
21                 (println "Fondos insuficientes")))))
22
23 ;; 100 transferencias concurrentes de 5 pesos cada una
24 (defn simular []
25     (let [fs (doall (repeatedly 100
26                         #(future (transferir cuenta-alice cuenta-bob 5)))))]
27         (doseq [f fs] @f)
28         (println "Alice:" @cuenta-alice ; esperado: 500
29         (println "Bob:"   @cuenta-bob   ; esperado: 1000
30         (println "Total:" (+ @cuenta-alice @cuenta-bob))) ; siempre 1500

```

```

31
32 (simular)

```

Listing 3: Transferencias bancarias con STM en Clojure. Ejecutar: `clojure -M -m banco.core`

Restricción importante: Las funciones pasadas a `alter` deben ser *puras* (sin efectos secundarios), porque el STM puede ejecutarlas múltiples veces si reintenta la transacción. Si pones un `println` dentro de un `dosync`, puede imprimirse más veces de las esperadas. Para efectos secundarios en transacciones, Clojure provee `io!` que lanza una excepción si se llama dentro de `dosync`.

4 Tabla Comparativa

Aspecto	Java	Rust	Clojure
Heap compartido	Sí	Sí	Sí (JVM)
Recolector de basura	Sí (pausa GC)	No (Ownership)	Sí (JVM GC)
Candados explícitos	<code>synchronized</code> / RL	<code>Mutex<T></code>	No (STM)
<code>volatile</code> / fences	<code>volatile</code> keyword	<code>Ordering::*</code>	No necesario
Dato acoplado al lock	No	Sí (vive en Mutex)	Sí (en <code>ref</code>)
Modelo de consistencia	Linealizable (con sync)	Linealizable (con Mutex)	Linealizable (con <code>dosync</code>)
Data races posibles	Sí (runtime)	No (compilación)	No (inmutabilidad)
Deadlocks posibles	Sí	Sí (mult. Mutex)	No (STM reintenta)
Uso típico HPC	Estructuras de datos conc.	Sistemas bajo nivel	Sistemas financieros

Tabla 2: Comparativa de modelos de concurrencia. RL = `ReentrantLock`.

5 Ejercicios

Instrucciones:

- La práctica se resuelve en equipos de máximo 3 integrantes. Entrega un PDF con las respuestas teóricas y, para los ejercicios con código, una breve descripción de cada programa.
- Solo un integrante sube la práctica; los demás escriben un comentario privado con el nombre de quien entregó.
- El formato y el medio de entrega los indicará tu ayudante de laboratorio.
- Entornos: Java 21 LTS. Rust: instalar con
`curl -proto 'https' -tlsv1.2 -sSf https://sh.rustup.rs | sh`
Clojure: REPL en línea en <https://tryclojure.org> si no tienen instalación local.

Puntaje total: 100 Tiempo estimado: $\approx$ 1 hr 30 min

5.1 Ejercicio 1 — Rust: el compilador como árbitro

[1hr / 50 puntos]

Este ejercicio tiene dos partes. En la primera intentarás escribir código incorrecto en Rust y observarás cómo el compilador lo rechaza. En la segunda extenderás el contrato inteligente del Listing 2 de la práctica.

Parte A: el rechazo del compilador (20 pts)

- (a) El siguiente código intenta replicar la race condition del contador de la Práctica 1 en Rust. Cópialo en un archivo `race.rs`, intenta compilarlo con `rustc race.rs` y pega el error del compilador en tu respuesta.

```
1 use std::thread;
2
3 fn main() {
4     let mut counter: i64 = 0;    // variable en el stack del main
5     let mut handles = vec![];
6
7     for _ in 0..4 {
8         // Intentamos pasar una referencia mutable a otro hilo
9         handles.push(thread::spawn(|| {
10             for _ in 0..1_000_000 {
11                 counter += 1;    // <- el compilador rechaza esto
12             }
13         }));
14     }
15     for h in handles { h.join().unwrap(); }
16     println!("Contador: {}", counter);
17 }
```

Listing 4: Código con race condition — NO compila en Rust.

- (b) Lee el mensaje de error del compilador. Identifica las frases clave y tradúcelas al vocabulario del curso: ¿qué regla del sistema de Ownership/Borrowing se viola? ¿Cómo se relaciona esa regla con la definición formal de data race (dos accesos concurrentes al mismo dato, al menos uno de escritura, sin sincronización)?
- (c) Corrige el código usando `Arc<Mutex<i64>` para obtener el contador linealizable. Verifica que el resultado es siempre 4 000 000.

Parte B: extender el contrato (30 pts)

Extiende `TokenContract` del Listing 2 de la práctica añadiendo dos operaciones:

- `mint(addr: &str, amount: u64)` — crea `amount` nuevos tokens y los deposita en `addr`.
 - `burn(addr: &str, amount: u64) -> bool` — destruye `amount` tokens de `addr`. Retorna `false` si la cuenta no tiene fondos suficientes.
- (d) Lanza 10 hilos concurrentes que ejecuten en un bucle de 100 iteraciones: cada hilo elige aleatoriamente entre `transfer`, `mint` y `burn` con montos pequeños. Al terminar todos, verifica que se cumple el siguiente invariante:

```

1 // suma_balances_actual == suma_balances_inicial + total_minted -
  total_burned
2 //
3 // Para verificarlo, rastrear total_minted y total_burned.
4 // Agregar contadores atomicos al SmartContract:
5 //   minted: AtomicU64
6 //   burned: AtomicU64

```

Listing 5: Invariante del contrato.

- (e) Identifica el punto de linearización de `mint()` y de `burn()`. ¿En qué línea exacta del código ocurre? ¿Por qué el invariante se mantiene aunque haya 10 hilos modificando el estado simultáneamente?

5.2 Ejercicio 2 — Clojure: observar el STM en acción

[1hr min / 50 puntos]

En este ejercicio modificarás el código de Clojure para hacer observable el comportamiento interno del STM: cuántas veces reintentará transacciones, y qué pasa cuando introduces efectos secundarios dentro de un bloque transaccional.

Parte A: contando reintentos (25 pts)

- (a) Copia el código del Listing 3 de la práctica en un archivo (o en el REPL de <https://tryclojure.org>) y añade un contador de reintentos *fuera* del `dosync`:

```

1 (ns banco.core)
2
3 (def cuenta-alice (ref 1000))
4 (def cuenta-bob   (ref 500))
5 ;; atom: actualizacion atomica sin coordinacion con otros refs
6 (def intentos (atom 0))
7
8 (defn transferir [origen destino cantidad]
9   (dosync
10    (swap! intentos inc) ; <- DENTRO del dosync: corre en cada reintento
11    (let [saldo @origen]
12      (if (>= saldo cantidad)
13        (do (alter origen - cantidad)
14            (alter destino + cantidad))
15        nil))))
16
17 (defn simular [n monto]
18   (reset! intentos 0)
19   (let [fs (doall (repeatedly n
20                        #(future (transferir cuenta-alice cuenta-bob monto)))))]
21     (doseq [f fs] @f)
22     {:alice      @cuenta-alice
23      :bob        @cuenta-bob
24      :intentos   @intentos
25      :futuros    n}))
26
27 ;; Ejecuta con distintos niveles de contencion:
28 (println (simular 10 10))
29 (println (simular 100 10))
30 (println (simular 500 5))

```

Listing 6: Contador de reintentos STM.

(b) Ejecuta las tres llamadas a `simular` y registra los resultados. Para cada ejecución calcula:

- Número de transferencias que realmente ocurrieron (cuántas veces cambió el saldo de Alice).
- Número de intentos totales registrados.
- Promedio de intentos por transacción exitosa.

(c) ¿Por qué el número de intentos aumenta con más hilos concurrentes? Explica en términos del protocolo STM: ¿qué condición hace que una transacción se reintente? ¿Puede haber livelock (todas las transacciones se reintenten indefinidamente)? ¿El STM de Clojure garantiza progreso?

Parte B: efectos secundarios y pureza (25 pts)

(d) Modifica `transferir` para añadir un `println` dentro del `dosync`:

```

1 (defn transferir-con-print [origen destino cantidad]
2   (dosync
3     (println "ejecutando transaccion") ; efecto secundario en STM
4     (let [saldo @origen]
5       (if (>= saldo cantidad)
6         (do (alter origen - cantidad)
7             (alter destino + cantidad))
8         nil))))

```

Listing 7: Transferir con efecto secundario.

- (e) Ejecuta (`simular 50 10`) usando esta versión. Cuenta cuántas veces se imprime "ejecutando transaccion" y cuántas transferencias realmente ocurrieron (cambios en el saldo). ¿Son iguales? ¿Por qué?
- (f) Relaciona tu observación con el principio: *las funciones dentro de un dosync deben ser puras (sin efectos secundarios)*. ¿Qué problema concreto introduce `println` dentro de la transacción? ¿Cómo lo resolverías si realmente necesitaras registrar en un log cada transferencia exitosa?

Pista: Clojure tiene (`io! body`) — una macro que lanza una excepción si se llama dentro de un `dosync`. Prueba a envolver el `println` con (`io! (println ...)`) y observa qué pasa.

Referencias

- [HS08] Maurice Herlihy and Nir Shavit. *The Art of Multiprocessor Programming*. Morgan Kaufmann, 2008. — Cap. 5 (Consistency, §5.3 Quiescent Consistency).
- [Sha11] Nir Shavit. Data structures in the multicore age. *Commun. ACM*, 54(3):76–84, 2011.
- [Man05] Jeremy Manson, William Pugh, and Sarita V. Adve. The Java Memory Model. *POPL 2005*, pp. 378–391.
- [Alg14] Jade Alglave, Luc Maranget, and Michael Tautschnig. Herding Cats: Modelling, Simulation, Testing, and Data Mining for Weak Memory. *ACM TOPLAS*, 36(2):7:1–7:74, 2014.
- [Vio16] Paolo Viotti and Marko Vukolić. Consistency in Non-Transactional Distributed Storage Systems. *ACM Comput. Surv.*, 49(1), 2016.
- [Rust] The Rust Reference — Concurrency primitives. <https://doc.rust-lang.org/std/sync/>
- [Clj] Rich Hickey. Clojure Refs and Transactions. <https://clojure.org/reference/refs>